APM52009 – Machine Learning for Scientific Computing and Numerical Analysis

Week 3: Optimization

Victorita Dolean and Hadrien Montanelli

Abstract

This lecture surveys optimization methods for training neural networks, from gradient descent and its stochastic variants (including momentum and higher-order extensions) to the efficient computation of gradients via backpropagation. We present both algorithms and convergence theory, and discuss regularization techniques. We then examine *spectral bias*, the phenomenon whereby gradient-based training fits low-frequency components of a target function faster than high-frequency ones. By linking this effect to the eigenstructure of kernel operators and a finite-element analogy for ReLU activations, we show how spectral bias acts as an implicit regularizer that favors smooth, low-frequency solutions—a key insight for SciML.

Contents

1 Training with gradient descent **2**

1.1 Gradient descent . . . 3

1.2 Stochastic gradient descent . . . 4

1.3 Backpropagation . . . 6

2 Extensions **8**

3 Regularization **9**

3.1 Ridge regularization . . . 9

3.2 Lasso regularization . . . 9

4 Spectral bias and training dynamics **11**

4.1 Spectral bias . . . 11

4.2 Analysis for shallow networks . . . 12

5 Exercises **14**

6 Suggested reading **15**

1 Training with gradient descent

In the previous chapter, we studied the *approximation capabilities* of neural networks—how well a given architecture can represent a target function with suitable weights. We now turn to the *optimization problem*: determining the weights that best fit a given dataset.

Given training data $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$, the network parameters $\boldsymbol{w} \in \mathbb{R}^p$ are determined by minimizing a suitable empirical loss functional

$$J(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} \left|f_w(\boldsymbol{x}_i) - y_i\right|^2, \tag{1.1}$$

which measures the discrepancy between the network prediction $f_w(\boldsymbol{x}_i)$ and the observed value y_i.

To study optimization methods in a mathematically controlled setting, we will for now *abstract away* from the specific neural network structure and consider the minimization of a general objective $J : \mathbb{R}^p \to \mathbb{R}$. In particular, we restrict our analysis to *smooth* and *strongly convex* functions, for which convergence results of iterative algorithms such as gradient descent can be rigorously established.

A differentiable function $J : \mathbb{R}^p \to \mathbb{R}$ is said to be *strongly convex* if there exists a constant $C > 0$ such that

$$J(\boldsymbol{w}) \geq J(\boldsymbol{v}) + \nabla J(\boldsymbol{v})^\top(\boldsymbol{w} - \boldsymbol{v}) + \frac{C}{2}\|\boldsymbol{w} - \boldsymbol{v}\|^2 \quad \forall \boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^p \qquad (C\text{-strongly convex}), \tag{1.2}$$

and it has an L-*Lipschitz continuous gradient* if there exists a constant $L > 0$ such that

$$\|\nabla J(\boldsymbol{w}) - \nabla J(\boldsymbol{v})\| \leq L\|\boldsymbol{w} - \boldsymbol{v}\| \quad \forall \boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^p \qquad (L\text{-Lipschitz gradient}). \tag{1.3}$$

A strongly convex function admits a unique minimizer, and the constants C and L quantify, respectively, its curvature and smoothness.

Remark 1.1. In this chapter, $\|\cdot\|$ denotes the 2-norm unless stated otherwise.

Remark 1.2. While convex optimization offers a clear theoretical framework, neural network training is inherently nonconvex, with many local minima and saddle points.

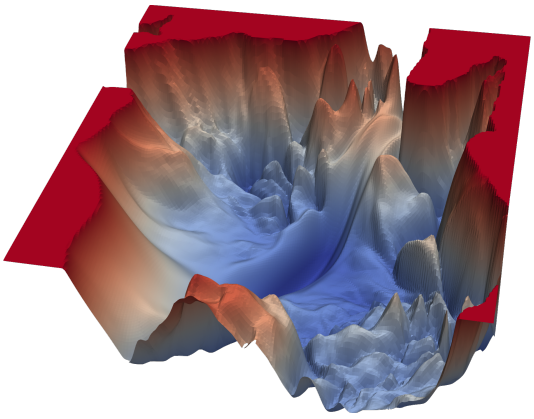


Figure 1: Typical optimization landscapes in machine learning are highly nonconvex.

We leave the following lemma, which is a direct consequence of the Lipschitz continuity of ∇J, as an exercise.

Lemma 1.3 (Consequence of L-Lipschitz gradient)**.** *Let $J : \mathbb{R}^p \to \mathbb{R}$ have an L-Lipschitz continuous gradient. Then*

$$J(\boldsymbol{w}) \leq J(\boldsymbol{v}) + \nabla J(\boldsymbol{v})^\top(\boldsymbol{w} - \boldsymbol{v}) + \frac{L}{2}\|\boldsymbol{w} - \boldsymbol{v}\|^2 \quad \forall \boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^p.$$

Combining Eq. (1.2) and Lemma 1.3, we obtain that if J is C-strongly convex with L-Lipschitz gradient, then necessarily $C \leq L$.

1.1 Gradient descent

To minimize $J(\boldsymbol{w})$, the simplest and most fundamental method is *gradient descent* (GD). At each step, the weights move opposite to the gradient of J, since $\nabla J(\boldsymbol{w})$ gives the local direction of steepest ascent. Subtracting a multiple of it drives the parameters toward smaller function values. For a suitable step size h, the objective decreases monotonically and the iterates converge to the minimizer. Convergence is guaranteed for C-strongly convex functions with L-Lipschitz gradients. With fixed $h > 0$, the algorithm is:

Algorithm 1 GD with fixed step size

1: **Input:** Initial guess $\boldsymbol{w}^0 \in \mathbb{R}^p$, step size $h > 0$.
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Compute the gradient $\nabla J(\boldsymbol{w}^k)$
4: Update $\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - h\,\nabla J(\boldsymbol{w}^k)$
5: **end for**

Lemma 1.4 (Descent property of GD)**.** *Let $J : \mathbb{R}^p \to \mathbb{R}$ have an L-Lipschitz continuous gradient. Then the GD iterates satisfy*

$$J(\boldsymbol{w}^{k+1}) - J(\boldsymbol{w}^k) \le -h\left(1 - \frac{hL}{2}\right)\|\nabla J(\boldsymbol{w}^k)\|^2.$$

Proof

From Lemma 1.3 and the update $\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - h\nabla J(\boldsymbol{w}^k)$, we directly obtain the desired inequality.

Theorem 1.5 (Convergence of GD with fixed step size)**.** *Let $J : \mathbb{R}^p \to \mathbb{R}$ be C-strongly convex with L-Lipschitz continuous gradient, and let $\boldsymbol{w}^*$ be its unique minimizer. Then, for any $h \in (0, 1/L]$, the GD iterates converge linearly to $\boldsymbol{w}^*$ and satisfy*

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 \le (1 - hC)^k\,\|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2, \quad J(\boldsymbol{w}^k) - J(\boldsymbol{w}^*) \le \frac{L}{2}(1 - hC)^k\,\|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2.$$

Proof

From $\boldsymbol{w}^k - \boldsymbol{w}^* = \boldsymbol{w}^{k-1} - \boldsymbol{w}^* - h\nabla J(\boldsymbol{w}^{k-1})$, we obtain

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 = \|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 - 2h\,\nabla J(\boldsymbol{w}^{k-1})^\top(\boldsymbol{w}^{k-1} - \boldsymbol{w}^*) + h^2\|\nabla J(\boldsymbol{w}^{k-1})\|^2.$$

By C-strong convexity,

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 \le (1 - hC)\|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 - 2h(J(\boldsymbol{w}^{k-1}) - J(\boldsymbol{w}^*)) + h^2\|\nabla J(\boldsymbol{w}^{k-1})\|^2.$$

From Lemma 1.4, we also have

$$h^2\|\nabla J(\boldsymbol{w}^{k-1})\|^2 \le \frac{h}{\left(1 - \frac{hL}{2}\right)}(J(\boldsymbol{w}^{k-1}) - J(\boldsymbol{w}^k)) \le \frac{h}{\left(1 - \frac{hL}{2}\right)}(J(\boldsymbol{w}^{k-1}) - J(\boldsymbol{w}^*)).$$

Hence

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 \le (1 - hC)\|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 + \left(\frac{h}{\left(1 - \frac{hL}{2}\right)} - 2h\right)(J(\boldsymbol{w}^{k-1}) - J(\boldsymbol{w}^*)).$$

The last term is nonpositive if and only if $h \in [0, 1/L]$, which gives

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 \le (1 - hC)\|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2.$$

Since $C > 0$ and $h \in (0, 1/L]$, the factor $(1 - hC) \in [0, 1)$, implying linear convergence. The second inequality follows directly from Lemma 1.3 and $\nabla J(\boldsymbol{w}^*) = 0$.

Remark 1.6 (Curvature and conditioning)**.** The constants C and L bound the curvature of J from below and above. When J is twice differentiable, these properties are equivalent to

$$CI_p \preceq \nabla^2 J(\boldsymbol{w}) \preceq LI_p \qquad \forall \boldsymbol{w} \in \mathbb{R}^p,$$

so that C and L are the smallest and largest eigenvalues of the Hessian. The ratio $\kappa = L/C$ plays the role of a *condition number*: when $\kappa \approx 1$, the problem is *well conditioned* and GD converges quickly; for large κ, it is *ill conditioned*, with elongated level sets and slow convergence. Thus, κ measures the anisotropy of the curvature of J near its minimum.

1.2 Stochastic gradient descent

The deterministic GD algorithm requires computing the full gradient $\nabla J(\boldsymbol{w})$ of the empirical loss

$$J(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} J_i(\boldsymbol{w}), \qquad J_i(\boldsymbol{w}) = \left|f_w(\boldsymbol{x}_i) - y_i\right|^2, \tag{1.4}$$

at each step, which is often infeasible when the number of parameters p and samples n is very large. This motivates stochastic approximations, where only a few samples are used per iteration. This relies on a simple idea: replace the exact gradient by an *unbiased stochastic estimator* built from one (or a few) randomly selected data points. This yields the *stochastic gradient descent* (SGD) method, originating in 1951 [14].

Algorithm 2 SGD with fixed step size

1: **Input:** Initial guess $\boldsymbol{w}^0 \in \mathbb{R}^p$, step size $h > 0$
2: **for** $k = 0, 1, 2, \ldots$ **do**
3: Sample an index $i_k \in \{1, \ldots, n\}$ uniformly at random
4: Compute the stochastic gradient $\boldsymbol{g}^k \leftarrow \nabla J_{i_k}(\boldsymbol{w}^k)$ ▷ so that $\mathbb{E}[\boldsymbol{g}^k|\boldsymbol{w}^k] = \nabla J(\boldsymbol{w}^k)$
5: Update $\boldsymbol{w}^{k+1} \leftarrow \boldsymbol{w}^k - h\,\boldsymbol{g}^k$
6: **end for**

More generally, we consider any estimator $\boldsymbol{g}^k$ that is *unbiased* for the true gradient at $\boldsymbol{w}^k$:

$$\mathbb{E}[\boldsymbol{g}^k \mid \boldsymbol{w}^k] = \nabla J(\boldsymbol{w}^k).$$

The iteration then reads

$$\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - h\,\boldsymbol{g}^k.$$

This defines a Markov chain $(\boldsymbol{w}^k)_{k\geq 0}$. For any random variable X, we write $\mathbb{E}_k[X] := \mathbb{E}[X|\boldsymbol{w}^k]$ for conditional expectation given the current iterate. The next lemma mirrors the deterministic descent inequality.

Lemma 1.7 (Descent property of SGD)**.** *Let $J : \mathbb{R}^p \to \mathbb{R}$ have an L-Lipschitz continuous gradient. Then the SGD iterates satisfy*

$$\mathbb{E}_k[J(\boldsymbol{w}^{k+1})] - J(\boldsymbol{w}^k) \;\leq\; -h\Big(\|\nabla J(\boldsymbol{w}^k)\|^2 - \frac{hL}{2}\,\mathbb{E}_k[\|\boldsymbol{g}^k\|^2]\Big).$$

Proof

By Lemma 1.3,

$$J(\boldsymbol{w}^{k+1}) \leq J(\boldsymbol{w}^k) + \nabla J(\boldsymbol{w}^k)^\top(\boldsymbol{w}^{k+1} - \boldsymbol{w}^k) + \frac{L}{2}\,\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^k\|^2. \tag{1.5}$$

Substituting $\boldsymbol{w}^{k+1} - \boldsymbol{w}^k = -h\,\boldsymbol{g}^k$ gives

$$J(\boldsymbol{w}^{k+1}) \leq J(\boldsymbol{w}^k) - h\,\nabla J(\boldsymbol{w}^k)^\top \boldsymbol{g}^k + \frac{h^2 L}{2}\,\|\boldsymbol{g}^k\|^2. \tag{1.6}$$

Taking $\mathbb{E}_k[\cdot]$ and using $\mathbb{E}_k[\boldsymbol{g}^k] = \nabla J(\boldsymbol{w}^k)$ yields the claim.

Motivated by Lemma 1.7, we assume a uniform second-moment bound: there exists $M > 0$ such that

$$\mathbb{E}_k[\|\boldsymbol{g}^k\|^2] \le M \qquad \text{for all } k \ge 0. \tag{1.7}$$

(One often uses $\mathbb{E}_k[\|\boldsymbol{g}^k\|^2] \le M + \|\nabla J(\boldsymbol{w}^k)\|^2$; we use the simpler uniform bound.)

Theorem 1.8 ("Convergence" of SGD with fixed step size)**.** *Let $J : \mathbb{R}^p \to \mathbb{R}$ be C-strongly convex with L-Lipschitz continuous gradient, and let $\boldsymbol{w}^*$ be its unique minimizer. Then, for any $h \in (0, 1/L]$, the SGD iterates satisfy*

$$\mathbb{E}[\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2] \le (1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2 + \frac{hM}{C},$$
$$\mathbb{E}[J(\boldsymbol{w}^k)] - J(\boldsymbol{w}^*) \le \frac{L}{2}(1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2 + \frac{hLM}{2C}.$$

Proof

As in the proof of Theorem 1.5,

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2 = \|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 - 2h\,\nabla J(\boldsymbol{w}^{k-1})^\top(\boldsymbol{w}^{k-1} - \boldsymbol{w}^*) + h^2\|\boldsymbol{g}^{k-1}\|^2. \tag{1.8}$$

Taking $\mathbb{E}_{k-1}[\cdot]$ and using $\mathbb{E}_{k-1}[\boldsymbol{g}^{k-1}] = \nabla J(\boldsymbol{w}^{k-1})$ gives

$$\mathbb{E}_{k-1}[\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2] = \|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 - 2h\,\nabla J(\boldsymbol{w}^{k-1})^\top(\boldsymbol{w}^{k-1} - \boldsymbol{w}^*) + h^2\mathbb{E}_{k-1}[\|\boldsymbol{g}^{k-1}\|^2]. \tag{1.9}$$

By C-strong convexity and Eq. (1.7),

$$\mathbb{E}_{k-1}[\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2] \le (1 - hC)\|\boldsymbol{w}^{k-1} - \boldsymbol{w}^*\|^2 + h^2 M. \tag{1.10}$$

Taking total expectation and unrolling the recursion yields

$$\mathbb{E}[\|\boldsymbol{w}^k - \boldsymbol{w}^*\|^2] \le (1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2 + h^2 M \sum_{j=0}^{k-1}(1 - hC)^j \le (1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|^2 + \frac{hM}{C}.$$

The bound on $\mathbb{E}[J(\boldsymbol{w}^k)] - J(\boldsymbol{w}^*)$ follows from Lemma 1.3 and $\nabla J(\boldsymbol{w}^*) = 0$.

Remark 1.9 (Diminishing step sizes)**.** With *diminishing* steps (h_k) satisfying $\sum_{k=0}^{\infty} h_k = \infty$ and $\sum_{k=0}^{\infty} h_k^2 < \infty$, and under standard moment assumptions on $\boldsymbol{g}^k$, one can strengthen Theorem 1.8 to conclude convergence to the minimizer in expectation (and, in some settings, almost surely). Constant steps yield a $\mathcal{O}(h)$ error; reducing h over time trades transient speed for vanishing asymptotic error.

Practical step-size scheduling Theorem 1.8 motivates a common heuristic: run SGD with a fixed step size until progress plateaus, then reduce h and continue. Let $h_0 \in (0, 1/L]$. Run SGD from $k_0 = 0$ to the first $k_1 > 0$ such that $\mathbb{E}[J(\boldsymbol{w}^{k_1})] - J(\boldsymbol{w}^*) \le 2F_{h_0}$, where $F_h := hLM/(2C)$ is the bound in Theorem 1.8. Then set $h_1 = h_0/2$ and repeat. This produces steps $h_r = h_0 2^{-r}$ and iterates k_r with $F_{h_r} \to 0$, and

$$\mathbb{E}[J(\boldsymbol{w}^{k_{r+1}})] - J(\boldsymbol{w}^*) \le 2F_{h_r} \qquad \forall\, r \ge 0, \tag{1.11}$$

so the expected suboptimality gap converges to zero.

Takeaways

GD moves opposite to the gradient to decrease the loss. For C-strongly convex J with L-Lipschitz gradient, linear convergence occurs for $h \in (0, 1/L]$, at a rate set by $\kappa = L/C$. **SGD** uses unbiased sample gradients:

- **Efficiency:** each step costs $\mathcal{O}(p)$ vs $\mathcal{O}(np)$ for full GD.
- **Steady-state error:** constant step size h yields $\mathcal{O}(h)$ oscillations around the minimum.
- **Diminishing steps:** decreasing h_k ensures convergence in expectation.

1.3 Backpropagation

Both gradient descent and its stochastic variants require efficient computation of ∇J. For neural networks with millions of parameters, this is achieved by *backpropagation*, which propagates sensitivities backward through the computational graph—essentially a clever implementation of the chain rule. It was popularized in [15] and is now the standard method for computing gradients in deep learning.

To illustrate, consider a single input–output pair $(\boldsymbol{x}, y)$ with the sample-wise loss

$$J(\boldsymbol{w}) = \left|f_w(\boldsymbol{x}) - y\right|^2 = \left|a^{(L+1)} - y\right|^2, \tag{1.12}$$

where the forward pass is defined as follows. Set $\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^{d_0}$ with $d_0 = d$; for $\ell = 1, \ldots, L$,

$$\boldsymbol{z}^{(\ell-1)} = W^{(\ell-1)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)} \in \mathbb{R}^{d_\ell}, \qquad \boldsymbol{a}^{(\ell)} = \sigma\Big(\boldsymbol{z}^{(\ell-1)}\Big) \in \mathbb{R}^{d_\ell}, \tag{1.13}$$

and at the output we take a linear readout with $d_{L+1} = 1$

$$a^{(L+1)} = z^{(L)} = W^{(L)}\boldsymbol{a}^{(L)} + b^{(L)} \in \mathbb{R}. \tag{1.14}$$

We use the Hadamard product $\odot$ and define, for $1 \le \ell \le L+1$,

$$\delta\boldsymbol{z}^{(\ell-1)} := \nabla_{\boldsymbol{z}^{(\ell-1)}} J \in \mathbb{R}^{d_\ell}, \qquad \delta\boldsymbol{b}^{(\ell-1)} := \nabla_{\boldsymbol{b}^{(\ell-1)}} J \in \mathbb{R}^{d_\ell}, \qquad \delta W^{(\ell-1)} := \nabla_{W^{(\ell-1)}} J \in \mathbb{R}^{d_\ell \times d_{\ell-1}}, \tag{1.15}$$

with the convention that $\delta z^{(L)}$ and $\delta b^{(L)}$ are scalars.

Algorithm 3 Forward and backward pass in backpropagation

Input: Sample $(\boldsymbol{x}, y)$, current parameters $\{W^{(\ell)}, \boldsymbol{b}^{(\ell)}\}_{\ell=0}^{L}$.
Forward pass: Set $\boldsymbol{a}^{(0)} = \boldsymbol{x}$.
for $\ell = 1$ to L **do**
  $\boldsymbol{z}^{(\ell-1)} \leftarrow W^{(\ell-1)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)}, \quad \boldsymbol{a}^{(\ell)} \leftarrow \sigma\Big(\boldsymbol{z}^{(\ell-1)}\Big)$.
end for
$a^{(L+1)} \leftarrow z^{(L)} \leftarrow W^{(L)}\boldsymbol{a}^{(L)} + b^{(L)}$.
Backward pass: $\delta z^{(L)} \leftarrow 2\big(a^{(L+1)} - y\big); \quad \delta b^{(L)} \leftarrow \delta z^{(L)}; \quad \delta W^{(L)} \leftarrow \delta z^{(L)}\big(\boldsymbol{a}^{(L)}\big)^\top$.
for $\ell = L$ down to 1 **do**
  $\delta\boldsymbol{z}^{(\ell-1)} \leftarrow \sigma'\Big(\boldsymbol{z}^{(\ell-1)}\Big) \odot \big(W^{(\ell)}\big)^\top \delta\boldsymbol{z}^{(\ell)}$.
  $\delta\boldsymbol{b}^{(\ell-1)} \leftarrow \delta\boldsymbol{z}^{(\ell-1)}, \quad \delta W^{(\ell-1)} \leftarrow \delta\boldsymbol{z}^{(\ell-1)}\big(\boldsymbol{a}^{(\ell-1)}\big)^\top$.
end for
Output: Gradients $\{\delta W^{(\ell)}, \delta\boldsymbol{b}^{(\ell)}\}_{\ell=0}^{L}$.

Before we state the main theorem, let us consider a shallow network $(L = 1)$ and illustrate these formulas. The forward (left) and backward (right) passes are given by

$$\begin{aligned}
&\boldsymbol{a}^{(0)} = \boldsymbol{x} \in \mathbb{R}^d, && \delta z^{(1)} = 2(a^{(2)} - y) \in \mathbb{R},\\
&\boldsymbol{z}^{(0)} = W^{(0)}\boldsymbol{a}^{(0)} + \boldsymbol{b}^{(0)} \in \mathbb{R}^{d_1}, && \delta b^{(1)} = \delta z^{(1)} \in \mathbb{R},\\
&\boldsymbol{a}^{(1)} = \sigma(\boldsymbol{z}^{(0)}) \in \mathbb{R}^{d_1}, && \delta W^{(1)} = \delta z^{(1)}(\boldsymbol{a}^{(1)})^\top \in \mathbb{R}^{1\times d_1},\\
&z^{(1)} = W^{(1)}\boldsymbol{a}^{(1)} + b^{(1)} \in \mathbb{R}, && \delta\boldsymbol{z}^{(0)} = \sigma'(\boldsymbol{z}^{(0)}) \odot (W^{(1)})^\top \delta z^{(1)} \in \mathbb{R}^{d_1},\\
&a^{(2)} = z^{(1)} \in \mathbb{R}, && \delta\boldsymbol{b}^{(0)} = \delta\boldsymbol{z}^{(0)} \in \mathbb{R}^{d_1},\\
& && \delta W^{(0)} = \delta\boldsymbol{z}^{(0)}(\boldsymbol{a}^{(0)})^\top \in \mathbb{R}^{d_1\times d}.
\end{aligned}$$

Theorem 1.10 (Backpropagation)**.** *The error vectors propagate backward as*

$$\delta z^{(L)} = 2\big(a^{(L+1)} - y\big), \qquad \delta\boldsymbol{z}^{(\ell-1)} = \sigma'\Big(\boldsymbol{z}^{(\ell-1)}\Big) \odot \big(W^{(\ell)}\big)^\top \delta\boldsymbol{z}^{(\ell)}, \quad 1 \le \ell \le L,$$

and the gradients are

$$\delta\boldsymbol{b}^{(\ell-1)} = \delta\boldsymbol{z}^{(\ell-1)}, \qquad \delta W^{(\ell-1)} = \delta\boldsymbol{z}^{(\ell-1)}\big(\boldsymbol{a}^{(\ell-1)}\big)^\top, \qquad 1 \le \ell \le L+1.$$

Proof

From Eq. (1.12) and $a^{(L+1)} = z^{(L)}$, we obtain

$$\delta z^{(L)} = \frac{\partial J}{\partial z^{(L)}} = \frac{\partial J}{\partial a^{(L+1)}} = 2(a^{(L+1)} - y).$$

For $1 \le \ell \le L$, we apply the chain rule to $\boldsymbol{z}^{(\ell)} = W^{(\ell)}\sigma(\boldsymbol{z}^{(\ell-1)}) + \boldsymbol{b}^{(\ell)}$:

$$\delta \boldsymbol{z}^{(\ell-1)} = \frac{\partial J}{\partial \boldsymbol{z}^{(\ell-1)}} = \left(\frac{\partial \boldsymbol{z}^{(\ell)}}{\partial \boldsymbol{z}^{(\ell-1)}}\right)^\top \frac{\partial J}{\partial \boldsymbol{z}^{(\ell)}} = \left(\frac{\partial \boldsymbol{z}^{(\ell)}}{\partial \boldsymbol{z}^{(\ell-1)}}\right)^\top \delta \boldsymbol{z}^{(\ell)}. \tag{1.16}$$

The Jacobian $\partial \boldsymbol{z}^{(\ell)}/\partial \boldsymbol{z}^{(\ell-1)}$ has entries

$$\left(\frac{\partial \boldsymbol{z}^{(\ell)}}{\partial \boldsymbol{z}^{(\ell-1)}}\right)_{ij} = W_{ij}^{(\ell)}\,\sigma'(\boldsymbol{z}^{(\ell-1)})_j \Rightarrow \left(\frac{\partial \boldsymbol{z}^{(\ell)}}{\partial \boldsymbol{z}^{(\ell-1)}}\right)^\top = \operatorname{diag}\!\big(\sigma'(\boldsymbol{z}^{(\ell-1)})\big)\,(W^{(\ell)})^\top,$$

which gives

$$\delta \boldsymbol{z}^{(\ell-1)} = \sigma'(\boldsymbol{z}^{(\ell-1)}) \odot (W^{(\ell)})^\top \delta \boldsymbol{z}^{(\ell)}.$$

Next, from $\boldsymbol{z}^{(\ell-1)} = W^{(\ell-1)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)}$, we have $\partial \boldsymbol{z}^{(\ell-1)}/\partial \boldsymbol{b}^{(\ell-1)} = I$, and hence

$$\delta \boldsymbol{b}^{(\ell-1)} = \left(\frac{\partial \boldsymbol{z}^{(\ell-1)}}{\partial \boldsymbol{b}^{(\ell-1)}}\right)^\top \frac{\partial J}{\partial \boldsymbol{z}^{(\ell-1)}} = I\,\delta \boldsymbol{z}^{(\ell-1)} = \delta \boldsymbol{z}^{(\ell-1)}.$$

Finally, for the weight matrix $W^{(\ell-1)}$, we have

$$(\delta W^{(\ell-1)})_{ij} = \frac{\partial J}{\partial W_{ij}^{(\ell-1)}} = \left(\frac{\partial J}{\partial \boldsymbol{z}^{(\ell-1)}}\right)^\top \frac{\partial \boldsymbol{z}^{(\ell-1)}}{\partial W_{ij}^{(\ell-1)}}.$$

Since $\boldsymbol{z}^{(\ell-1)} = W^{(\ell-1)}\boldsymbol{a}^{(\ell-1)} + \boldsymbol{b}^{(\ell-1)}$, the k-th component of $\partial \boldsymbol{z}^{(\ell-1)}/\partial W_{ij}^{(\ell-1)}$ is

$$\left(\frac{\partial \boldsymbol{z}^{(\ell-1)}}{\partial W_{ij}^{(\ell-1)}}\right)_k = (\boldsymbol{a}^{(\ell-1)})_j\,\delta_{ik},$$

so that

$$(\delta W^{(\ell-1)})_{ij} = (\delta \boldsymbol{z}^{(\ell-1)})_i\,(\boldsymbol{a}^{(\ell-1)})_j,$$

which, in matrix form, gives $\delta W^{(\ell-1)} = \delta \boldsymbol{z}^{(\ell-1)}(\boldsymbol{a}^{(\ell-1)})^\top$.

Theorem 1.10 shows that all parameter gradients can be computed recursively from the output layer backward. Reusing forward-pass intermediates makes the cost of ∇J roughly the same as evaluating $f_w(\boldsymbol{x})$.

Remark 1.11 (Vectorized view and mini-batches)**.** For a mini-batch $\{(\boldsymbol{x}_j, y_j)\}_{j=1}^B$, the same formulas hold sample-wise. Averaging over the batch reduces variance and integrates seamlessly with SGD:

$$\delta W^{(\ell)} = \frac{1}{B}\sum_{j=1}^{B} \delta \boldsymbol{z}_{(j)}^{(\ell)} (\boldsymbol{a}_{(j)}^{(\ell)})^\top, \quad \delta \boldsymbol{b}^{(\ell)} = \frac{1}{B}\sum_{j=1}^{B} \delta \boldsymbol{z}_{(j)}^{(\ell)}.$$

Takeaways

Backpropagation efficiently computes gradients in feedforward networks:

- Gradients are propagated backward from the output layer using the chain rule.
- Forward quantities are reused, making the cost of ∇J roughly that of one forward evaluation.
- Naturally extends to mini-batches: compute per-sample gradients and average for SGD.

Lesson for SciML: SGD is the practical workhorse of large-scale learning—simple, noisy, but effective when paired with good step-size scheduling and variance-reduction techniques.

2 Extensions

Variable step size Fixed-step gradient methods can be generalized to variable step sizes $(h_k)_{k\geq 0}$:

- GD: one can use a *line search* to select h_k at each step. This is useful when the constant L or C is unknown, avoiding steps that are too large (which risk divergence) or too small (which slow convergence).
- SGD: one can use a *diminishing step size*, e.g., $h_k = h_0/(k+1)$. This gradually reduces the noise in the stochastic gradients, allowing iterates to settle near the minimum while maintaining initial progress.

Higher-order methods Newton-type methods exploit second-order information (Hessian) for faster, often quadratic convergence near the minimum. Quasi-Newton methods such as BFGS approximate the Hessian, reducing computational cost while retaining superlinear convergence. See Algorithm 4 and Algorithm 5.

Algorithm 4 Newton's Method

Input: Initial guess $\boldsymbol{w}^0 \in \mathbb{R}^p$
for $k = 0, 1, 2, \ldots$ **do**
  Compute $\nabla J(\boldsymbol{w}^k)$ and $\nabla^2 J(\boldsymbol{w}^k)$
  Solve $\nabla^2 J(\boldsymbol{w}^k)\, \boldsymbol{s}^k = -\nabla J(\boldsymbol{w}^k)$ ▷ matrix inversion/linear solve $\mathcal{O}(p^3)$
  Update $\boldsymbol{w}^{k+1} \leftarrow \boldsymbol{w}^k + \boldsymbol{s}^k$
end for

Algorithm 5 BFGS

Input: Initial guess $\boldsymbol{w}^0 \in \mathbb{R}^p$, initial inverse Hessian approximation $H^0 \in \mathbb{R}^{p\times p}$
Compute $\nabla J(\boldsymbol{w}^0)$
for $k = 0, 1, 2, \ldots$ **do**
  Choose step size $\alpha^k > 0$ and compute $\boldsymbol{s}^k \leftarrow -\alpha^k H^k \nabla J(\boldsymbol{w}^k)$ ▷ matrix-vector multiply $\mathcal{O}(p^2)$
  Update $\boldsymbol{w}^{k+1} \leftarrow \boldsymbol{w}^k + \boldsymbol{s}^k$
  Compute $\nabla J(\boldsymbol{w}^{k+1})$ and set $\boldsymbol{y}^k \leftarrow \nabla J(\boldsymbol{w}^{k+1}) - \nabla J(\boldsymbol{w}^k)$
  Update

$$H^{k+1} \leftarrow H^k + \frac{[(\boldsymbol{s}^k)^\top \boldsymbol{y}^k + (\boldsymbol{y}^k)^\top H^k \boldsymbol{y}^k](\boldsymbol{s}^k(\boldsymbol{s}^k)^\top)}{\left((\boldsymbol{s}^k)^\top \boldsymbol{s}^k\right)^2}$$

end for

Momentum and adaptive methods These accelerate first-order optimization by accumulating past gradients. Adaptive methods like Adam combine momentum with per-parameter scaling to stabilize and speed up training.

Algorithm 6 Adam

Input: Initial guess $\boldsymbol{w}^0 \in \mathbb{R}^p$, step size $h > 0$, parameters $\epsilon > 0$, $\beta \in [0, 1)$, $\gamma \in [0, 1)$
Initialize $\boldsymbol{m}^0 = \boldsymbol{v}^0 = \boldsymbol{0} \in \mathbb{R}^p$
for $k = 0, 1, 2, \ldots$ **do**
  Compute the gradient $\nabla J(\boldsymbol{w}^k)$
  Update $\boldsymbol{m}^{k+1} \leftarrow \beta \boldsymbol{m}^k + (1-\beta)\nabla J(\boldsymbol{w}^k)$
  Update $\boldsymbol{v}^{k+1} \leftarrow \gamma \boldsymbol{v}^k + (1-\gamma)\nabla J(\boldsymbol{w}^k) \odot \nabla J(\boldsymbol{w}^k)$
  Update $\boldsymbol{w}^{k+1} \leftarrow \boldsymbol{w}^k - h\frac{\boldsymbol{m}^{k+1}}{1-\beta^{k+1}} \oslash \left\{\sqrt{\frac{\boldsymbol{v}^{k+1}}{1-\gamma^{k+1}}} + \epsilon\right\}$
end for

Takeaways

- Variable step sizes can improve convergence speed and stability.
- Higher-order methods (Newton, BFGS) exploit curvature for faster convergence but cost more.
- Momentum and adaptive methods (Adam) enhance first-order methods by damping oscillations and adjusting learning rates per parameter.

3 Regularization

3.1 Ridge regularization

Setup Let $J : \mathbb{R}^p \to \mathbb{R}$ be any loss. Ridge (or ℓ_2) regularization adds a differentiable penalty R to the loss:

$$J_R(\boldsymbol{w}) = J(\boldsymbol{w}) + R(\boldsymbol{w}) = J(\boldsymbol{w}) + \frac{\lambda}{2}\|\boldsymbol{w}\|_2^2, \qquad \lambda > 0.$$

Since the penalty is differentiable, we can compute the gradient

$$\nabla J_R(\boldsymbol{w}) = \nabla J(\boldsymbol{w}) + \lambda \boldsymbol{w},$$

so all the usual backpropagation machinery applies with a simple additive term.

Remark 3.1. Ridge regularization helps control overfitting, stabilize optimization in ill-conditioned problems, and reduce sensitivity to small singular values of the Jacobian. Moreover, if J is convex, then J_R is strongly convex since the ℓ_2 penalty is strongly convex.

3.2 Lasso regularization

Setup Let $J : \mathbb{R}^p \to \mathbb{R}$ be any loss. Lasso (or ℓ_1) regularization reads

$$J_R(\boldsymbol{w}) = J(\boldsymbol{w}) + R(\boldsymbol{w}) = J(\boldsymbol{w}) + \lambda\|\boldsymbol{w}\|_1,$$

for some regularization parameter $\lambda > 0$. Since J_R is not differentiable because of R, we cannot use GD. However, we follow the same idea as with GD by considering a "quadratic plus R" model:

$$\begin{aligned}\boldsymbol{w}^{k+1} &= \arg\min_{\boldsymbol{z}\in\mathbb{R}^p} \left\{ \nabla J(\boldsymbol{w}^k)^\top(\boldsymbol{z} - \boldsymbol{w}^k) + \frac{1}{2h}\|\boldsymbol{z} - \boldsymbol{w}^k\|_2^2 + R(\boldsymbol{z}) \right\}, \\ &= \arg\min_{\boldsymbol{z}\in\mathbb{R}^p} \left\{ \frac{1}{2h} \left\| \boldsymbol{z} - (\boldsymbol{w}^k - h\nabla J(\boldsymbol{w}^k)) \right\|_2^2 + R(\boldsymbol{z}) \right\}.\end{aligned}$$

Thus we can write the iteration as the proximal-gradient step

$$\boldsymbol{w}^{k+1} = \mathrm{prox}_{hR}(\boldsymbol{w}^k - h\nabla J(\boldsymbol{w}^k)),$$

where the proximal operator is defined for any $\boldsymbol{v} \in \mathbb{R}^p$ by

$$\mathrm{prox}_{hR}(\boldsymbol{w}) = \arg\min_{\boldsymbol{z}\in\mathbb{R}^d} \left\{ \frac{1}{2h}\|\boldsymbol{w} - \boldsymbol{z}\|_2^2 + R(\boldsymbol{z}) \right\}.$$

Equivalently, defining the generalized gradient mapping G_h gives

$$\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - h\, G_h(\boldsymbol{w}^k), \quad G_h(\boldsymbol{w}) = \frac{\boldsymbol{w} - \mathrm{prox}_{hR}(\boldsymbol{w} - h\nabla J(\boldsymbol{w}))}{h}.$$

The vector $G_h(\boldsymbol{w}) - \nabla J(\boldsymbol{w})$ is called a subgradient of $R(\boldsymbol{w})$ at the point $\mathrm{prox}_{hR}(\boldsymbol{w} - h\nabla J(\boldsymbol{w}))$.

Remark 3.2. Lasso promotes sparsity by driving small coefficients exactly to zero, which is useful for feature selection and interpretability. Since R is merely convex, unless J is strongly convex, J_R is only convex.

Convergence theory In the following, we assume that J is convex and L-smooth, and that R is convex with well-defined proximal operator.

Lemma 3.3 (Proximal Gradient Descent property)**.** *The proximal gradient descent iterates satisfy*

$$J_R(\boldsymbol{w}^{k+1}) - J_R(\boldsymbol{z}) \le -h\left(1 - \frac{hL}{2}\right)\|G_h(\boldsymbol{w}^k)\|_2^2 + G_h(\boldsymbol{w}^k)^\top(\boldsymbol{w}^k - \boldsymbol{z}),$$

for all $\boldsymbol{z} \in \mathbb{R}^p$. In particular, for $\boldsymbol{z} = \boldsymbol{w}^k$,

$$J_R(\boldsymbol{w}^{k+1}) - J_R(\boldsymbol{w}^k) \le -h\left(1 - \frac{hL}{2}\right)\|G_h(\boldsymbol{w}^k)\|_2^2.$$

Proof

Let $\boldsymbol{z} \in \mathbb{R}^d$. Via convexity of J,

$$J(\boldsymbol{z}) \geq J(\boldsymbol{w}^k) + \nabla J(\boldsymbol{w}^k)^\top (\boldsymbol{z} - \boldsymbol{w}^k).$$

Moreover, via L-smoothness,

$$J(\boldsymbol{w}^{k+1}) \leq J(\boldsymbol{w}^k) + \nabla J(\boldsymbol{w}^k)^\top (\boldsymbol{w}^{k+1} - \boldsymbol{w}^k) + \frac{L}{2}\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^k\|_2^2.$$

Together this yields

$$J(\boldsymbol{w}^{k+1}) \leq J(\boldsymbol{z}) + \nabla J(\boldsymbol{w}^k)^\top (\boldsymbol{w}^{k+1} - \boldsymbol{w}^k) - \nabla J(\boldsymbol{w}^k)^\top (\boldsymbol{z} - \boldsymbol{w}^k) + \frac{L}{2}\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^k\|_2.$$

Now, via convexity of R,

$$R(\boldsymbol{z}) \geq R(\boldsymbol{w}^{k+1}) + g(\boldsymbol{w}^{k+1})^\top (\boldsymbol{z} - \boldsymbol{w}^{k+1}),$$

for any subgradient g at $\boldsymbol{w}^{k+1}$. In particular, for $G_h(\boldsymbol{w}^k) - \nabla J(\boldsymbol{w}^k)$,

$$R(\boldsymbol{w}^{k+1}) \leq R(\boldsymbol{z}) - (G_h(\boldsymbol{w}^k) - \nabla J(\boldsymbol{w}^k))^\top (\boldsymbol{z} - \boldsymbol{w}^{k+1}).$$

Combining the inequalities for J and R, we get

$$J_R(\boldsymbol{w}^{k+1}) \leq J_R(\boldsymbol{z}) + G_h(\boldsymbol{w}^k)^\top (\boldsymbol{w}^{k+1} - \boldsymbol{z}) + \frac{L}{2}\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^k\|_2^2.$$

We now use the iteration $\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - hG_h(\boldsymbol{w}^k)$, which leads to

$$\begin{aligned} J_R(\boldsymbol{w}^{k+1}) - J_R(\boldsymbol{z}) &\leq G_h(\boldsymbol{w}^k)^\top (\boldsymbol{w}^k - hG_h(\boldsymbol{w}^k) - \boldsymbol{z}) + \frac{h^2 L}{2}\|G_h(\boldsymbol{w}^k)\|_2^2, \\ &= -h\left(1 - \frac{hL}{2}\right)\|G_h(\boldsymbol{w}^k)\|_2^2 + G_h(\boldsymbol{w}^k)^\top (\boldsymbol{w}^k - \boldsymbol{z}). \end{aligned}$$

Theorem 3.4 (Convergence of Proximal Gradient Descent)**.** *If J_R is C-strongly convex, then proximal gradient descent converges to $\boldsymbol{w}^*$ and*

$$\|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2 \leq (1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|_2^2, \quad J_R(\boldsymbol{w}^k) - J_R(\boldsymbol{w}^*) \leq \frac{L}{2}(1 - hC)^k \|\boldsymbol{w}^0 - \boldsymbol{w}^*\|_2^2.$$

Proof

We utilize the PGD property with $\boldsymbol{z} = \boldsymbol{w}^*$ and $h \leq 1/L$,

$$\begin{aligned} J_R(\boldsymbol{w}^{k+1}) - J_R(\boldsymbol{w}^*) &\leq G_h(\boldsymbol{w}^k)^\top (\boldsymbol{w}^k - \boldsymbol{w}^*) - \frac{h}{2}\|G_h(\boldsymbol{w}^k)\|_2^2, \\ &= \frac{1}{2h}\left(2hG_h(\boldsymbol{w}^k)^\top (\boldsymbol{w}^k - \boldsymbol{w}^*) - \|hG_h(\boldsymbol{w}^k)\|_2^2\right), \\ &= \frac{1}{2h}\left(\|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2 - \|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2\right). \end{aligned}$$

In the C-strongly convex case, we rewrite the inequality as

$$\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2 \leq \|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2 - 2h(J_R(\boldsymbol{w}^{k+1}) - J_R(\boldsymbol{w}^*)),$$

and utilize the fact that

$$J_R(\boldsymbol{w}^{k+1}) - J(\boldsymbol{w}^*) \geq \frac{C}{2}\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2.$$

This yields

$$\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2 \leq \|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2 - hC\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2,$$

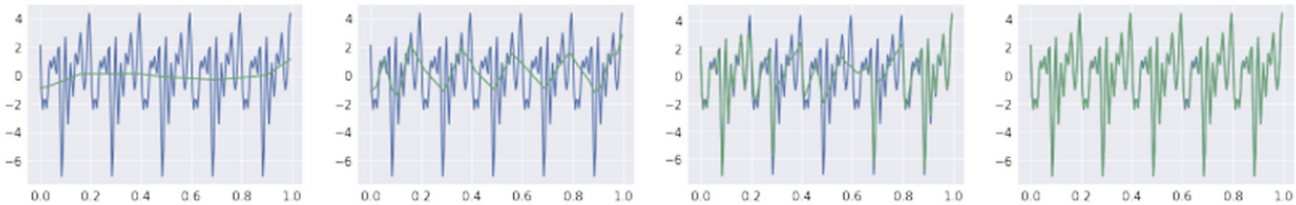
and hence

$$\|\boldsymbol{w}^{k+1} - \boldsymbol{w}^*\|_2^2 \leq (1 + hC)^{-1}\|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2 \leq (1 - hC)\|\boldsymbol{w}^k - \boldsymbol{w}^*\|_2^2.$$

4 Spectral bias and training dynamics

4.1 Spectral bias

An intriguing empirical observation, first noted in [13], is that neural networks trained by gradient-based optimization do not learn all frequency components of a target function at the same speed. Instead, the low-frequency (smooth) modes of the signal are fitted first, while high-frequency (less smooth) oscillations appear later in training. This phenomenon is known as the *spectral bias* or *frequency bias* of neural networks.



(a) Iteration 100 (b) Iteration 1000 (c) Iteration 10000 (d) Iteration 80000

Figure 2: Illustration of *spectral bias* during neural network training, from [13]. The blue curve represents the target function, while the green curve denotes the network output at different training iterations. Early in training the network captures only the low-frequency, smooth components of the signal.

The notion of *frequency*, central to spectral bias, aligns with Fourier modes in one dimension and is rigorously defined via the eigenfunctions of a compact self-adjoint operator in high-dimensional or discrete settings.

Continuous notion of frequency Consider the convolution operator $T_s : L^2(\mathbb{R}^d) \to L^2(\mathbb{R}^d)$ defined by

$$(T_s f)(\boldsymbol{x}) = \left(\frac{s}{2\pi}\right)^{d/2} \int_{\mathbb{R}^d} e^{-\frac{s}{2}\|\boldsymbol{x}-\boldsymbol{y}\|^2} f(\boldsymbol{y})\, d\boldsymbol{y}. \tag{4.1}$$

This operator averages f with a Gaussian kernel of variance $\sim 1/s$ and equivalently represents the action of the heat semigroup at time $t = s^{-1}$: if u solves

$$\partial_t u - \frac{1}{2}\Delta u = 0, \qquad u(0, \boldsymbol{x}) = f(\boldsymbol{x}),$$

then $u(s^{-1}, \boldsymbol{x}) = T_s f(\boldsymbol{x})$. Thus, T_s is a *smoothing operator*. Its eigenfunctions and eigenvalues are

$$\phi_{\boldsymbol{\omega}}(\boldsymbol{x}) = e^{i\boldsymbol{\omega}\cdot\boldsymbol{x}}, \quad \mu_{\boldsymbol{\omega}} = e^{-\|\boldsymbol{\omega}\|^2/(2s)}, \qquad \boldsymbol{\omega} \in \mathbb{R}^d.$$

High-frequency modes (large $\|\boldsymbol{\omega}\|$) decay faster under T_s, while low-frequency modes are preserved. Frequency can thus be interpreted as the inverse magnitude of the eigenvalue of this smoothing operator.

Discrete notion of frequency For data sampled at finitely many points $\mathcal{D} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\} \subset \mathbb{R}^d$, the classical Fourier modes are no longer directly applicable, especially in high dimensions. A natural generalization is obtained by defining *frequency* through the eigenfunctions of a positive definite kernel on the dataset.

Let $X = L^2(\mathcal{D})$ denote the space of square-summable functions on $\mathcal{D}$, and define $K : X \to X$ by

$$(Kf)(\boldsymbol{x}_i) = \sum_{j=1}^{N} k(\boldsymbol{x}_i, \boldsymbol{x}_j)\, f(\boldsymbol{x}_j), \tag{4.2}$$

where $k : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$ is symmetric and positive definite (e.g., the Gaussian RBF kernel $k(\boldsymbol{x}, \boldsymbol{y}) = e^{-\sigma\|\boldsymbol{x}-\boldsymbol{y}\|^2}$). Since K is self-adjoint and positive semidefinite, it admits an orthonormal set of eigenfunctions $\{\phi_1, \ldots, \phi_N\}$:

$$K\phi_i = \mu_i \phi_i, \qquad \mu_1 \geq \mu_2 \geq \cdots \geq 0.$$

These ϕ_i play the role of *discrete frequencies* adapted to the data geometry. Eigenfunctions with large eigenvalues correspond to slowly varying, smooth patterns across the dataset, whereas those with small eigenvalues capture rapidly oscillating components.

Remark 4.1. This operator-based view provides a unified notion of frequency that applies both to continuous domains and to high-dimensional, finite datasets. It connects naturally to kernel methods, diffusion maps, and graph Laplacians: the eigenfunctions of these operators serve as analogues of Fourier modes, defining a frequency ordering independent of coordinate systems.

Definition 4.2 (Spectral bias)**.** Let $f^{(\ell)}$ be the network at iteration ℓ, and f^* the empirical minimizer. Expand the difference in the ϕ_j basis:

$$f^{(\ell)} - f^* = \sum_j e_j^{(\ell)} \phi_j.$$

Spectral bias is the observation that coefficients associated with larger eigenvalues converge faster:

$|e_j^{(\ell)}|$ decays faster for large μ_i (low frequencies) than for small μ_i (high frequencies).

4.2 Analysis for shallow networks

A clear analysis is given in [7]. Let $f : [0,1] \to \mathbb{R}$ be a target function and consider a shallow neural network:

$$f_w(x) = \sum_{i=1}^{n} w_i \sigma_i(x), \quad \sigma_i(x) = \sigma\left(x - \frac{i-1}{n}\right), \quad \sigma(x) = \max(0, x), \tag{4.3}$$

where only the outer weights $\boldsymbol{w} = (w_1, \ldots, w_n)^\top$ are trainable. The loss functional is the mean square error

$$J(\boldsymbol{w}) = \frac{1}{2} \int_0^1 |f(x) - f_w(x)|^2 dx. \tag{4.4}$$

Lemma 4.3 (Structure of the loss functional)**.** *The loss in Eq. (4.4) can be written in quadratic form as*

$$J(\boldsymbol{w}) = \tfrac{1}{2} \boldsymbol{w}^\top M \boldsymbol{w} - \boldsymbol{b}^\top \boldsymbol{w} + c,$$

where the SPD mass matrix $M \in \mathbb{R}^{n \times n}$*, the load vector* $\boldsymbol{b} \in \mathbb{R}^n$ *and* $c \in \mathbb{R}$ *are defined by*

$$M_{ij} = \int_0^1 \sigma_i(x)\sigma_j(x) dx, \quad b_i = \int_0^1 f(x)\sigma_i(x) dx, \quad c = \frac{1}{2} \int_0^1 f(x)^2 dx. \tag{4.5}$$

Proof

Expanding Eq. (4.4) and using the linearity of the integral gives

$$J(\boldsymbol{w}) = \frac{1}{2} \int_0^1 f(x)^2 dx - \int_0^1 f(x) \sum_{i=1}^{n} w_i \sigma_i(x) dx + \frac{1}{2} \int_0^1 \left(\sum_{i=1}^{n} w_i \sigma_i(x) \right)^2 dx.$$

Identifying the second and third terms with the definitions in Eq. (4.5) yields the stated expression. Positivity follows from the fact that the functions $\{\sigma_i(x)\}_{i=1}^n$ are linearly independent on $[0,1]$, hence the matrix of their L^2 inner products is positive definite.

Gradient descent Consider the gradient descent update on $J(\boldsymbol{w})$ with step size $h > 0$:

$$\boldsymbol{w}^{(\ell+1)} = \boldsymbol{w}^{(\ell)} - h \nabla J(\boldsymbol{w}^{(\ell)}) = \boldsymbol{w}^{(\ell)} - h(M\boldsymbol{w}^{(\ell)} - \boldsymbol{b}).$$

Let $\boldsymbol{w}^* = M^{-1}\boldsymbol{b}$ be the minimizer, and define the error $\boldsymbol{e}^{(\ell)} = \boldsymbol{w}^{(\ell)} - \boldsymbol{w}^*$. Then

$$\boldsymbol{e}^{(\ell+1)} = (I - hM)\, \boldsymbol{e}^{(\ell)}. \tag{4.6}$$

Let $0 \le \lambda_1 \le \cdots \le \lambda_n$ denote the eigenvalues of M and $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ the corresponding orthonormal eigenvectors. Expanding $\boldsymbol{e}^{(\ell)}$ in the eigenbasis of M gives

$$\boldsymbol{e}^{(\ell)} = \sum_{j=1}^{n} e_j^{(\ell)} \boldsymbol{v}_j \quad \Longrightarrow \quad e_j^{(\ell+1)} = (1 - h\lambda_j) e_j^{(\ell)}. \tag{4.7}$$

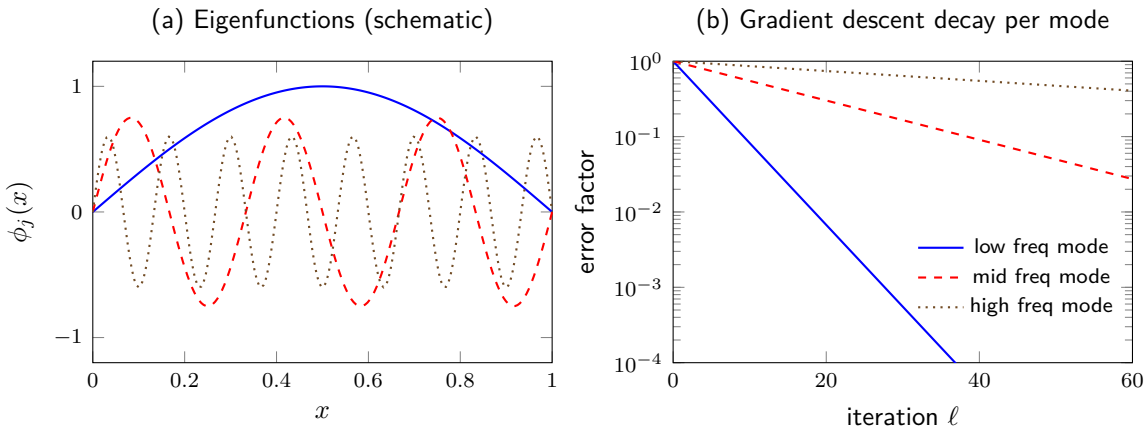


Figure 3: Spectral behaviour of gradient descent. (a) Schematic eigenfunctions: low-frequency modes are smooth; higher-index modes oscillate rapidly. (b) Gradient descent damps low-frequency components much faster than high-frequency ones, the essence of spectral bias.

From parameter space to function space Each eigenvector $\boldsymbol{v}_j$ of M defines a particular linear combination of ReLU basis functions in Eq. (4.3). We introduce the *eigenfunctions of the network basis*:

$$\phi_j(x) = \sum_{i=1}^{n} (\boldsymbol{v}_j)_i \sigma_i(x). \qquad 1 \le j \le n. \tag{4.8}$$

By construction, these functions are orthogonal in $L^2([0,1])$:

$$\langle \phi_j, \phi_k \rangle_{L^2([0,1])} = \boldsymbol{v}_j^\top M \boldsymbol{v}_k = \lambda_k \, \boldsymbol{v}_j^\top \boldsymbol{v}_k = 0 \quad (j \neq k).$$

Evolution of the network Let $f^{(\ell)}$ be the network at iteration ℓ and f^* the one with weights $\boldsymbol{w}^*$. Then

$$f^{(\ell)}(x) - f^*(x) = \sum_{i=1}^{n} \left(w_i^{(\ell)} - w_i^* \right) \sigma_i(x) = \sum_{i=1}^{n} \left[\sum_{j=1}^{n} e_j^{(\ell)} (\boldsymbol{v}_j)_i \right] \sigma_i(x) = \sum_{j=1}^{n} e_j^{(\ell)} \phi_j(x).$$

with, from Eq. (4.7),

$$e_j^{(\ell)} = (1 - h\lambda_j)^\ell e_j^{(0)},$$

so that components associated with large λ_j (smooth, low-frequency modes) converge faster than those with small λ_j (high-frequency modes). All this can be summarised in the following result:

Theorem 4.4 (Spectral decay of the training error)**.** *Assume the step size satisfies $0 < h \le 2/\lambda_n$. Then*

$$\|\boldsymbol{e}^{(\ell)}\|_2^2 = \|\boldsymbol{w}^{(\ell)} - \boldsymbol{w}^*\|_2^2 = \sum_{j=1}^{n} (1 - h\lambda_j)^{2\ell} \, |e_j^{(0)}|^2.$$

Moreover, the L^2 error of the network satisfies

$$\|f^{(\ell)} - f^*\|_{L^2([0,1])}^2 = \sum_{j=1}^{n} (1 - h\lambda_j)^{2\ell} |e_j^{(0)}|^2.$$

Finally, the following theorem shows that the eigenfunctions of M associated with large eigenvalues are smooth, whereas those associated with small eigenvalues are highly oscillatory (see Fig. 3).

Theorem 4.5 (ReLU frequency scaling)**.** *The eigenfunctions in Eq. (4.8) verify*

$$\frac{\int_0^1 |\phi_j'(x)|^2 \, dx}{\int_0^1 |\phi_j(x)|^2 \, dx} \asymp \frac{n^4}{j^4}.$$

In particular, large-λ_j eigenfunctions are smoother, while small-λ_j eigenfunctions are highly oscillatory.

5 Exercises

Exercise 5.1 (Proximal operator for the Lasso)**.** Consider the ℓ_1-regularized least-squares problem

$$J_R(\boldsymbol{w}) = J(\boldsymbol{w}) + R(\boldsymbol{w}) = \tfrac{1}{2}\|X\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda\|\boldsymbol{w}\|_1, \qquad \lambda > 0.$$

(a) Show that the proximal operator of $R(\boldsymbol{w}) = \lambda\|\boldsymbol{w}\|_1$ is given by

$$\operatorname{prox}_{\lambda h}(\boldsymbol{v}) = \arg\min_{\boldsymbol{z}\in\mathbb{R}^p} \Big\{ \frac{1}{2h}\|\boldsymbol{v} - \boldsymbol{z}\|_2^2 + \lambda\|\boldsymbol{z}\|_1 \Big\}.$$

(b) Derive its closed-form expression, known as the *soft-thresholding map* S_t with $t = \lambda h$:

$$(S_t(\boldsymbol{v}))_i = \begin{cases} v_i - t, & \text{if } v_i > t, \\ 0, & \text{if } v_i \in [-t, t], \\ v_i + t, & \text{if } v_i < -t. \end{cases}$$

(c) Deduce that the proximal gradient descent iteration takes the form

$$\boldsymbol{w}^{k+1} = S_{\lambda h}\Big(\boldsymbol{w}^k - hX^\top(X\boldsymbol{w}^k - \boldsymbol{y})\Big).$$

Exercise 5.2 (Mass matrix in the hat basis)**.** Consider the standard piecewise-linear finite element basis $\{\varphi_i\}_{i=1}^n$ on the grid $x_i = i/n$ over $(0,1)$. Let $M_{\text{hat}} \in \mathbb{R}^{n\times n}$ be the mass matrix with entries

$$(M_{\text{hat}})_{ik} := \int_0^1 \varphi_i(x)\varphi_k(x)\,dx.$$

(a) Compute the nonzero entries of M_{hat} and show that it is tridiagonal with a $(4,1,1)$ pattern.

(b) Use Gershgorin's theorem to show that $c\,h \le \lambda_{\min}(M_{\text{hat}}) \le \lambda_{\max}(M_{\text{hat}}) \le C\,h$ with $h = 1/n$ and constants c, C independent of n.

Exercise 5.3 (Change of basis between ReLU and hat functions)**.** Let $\{\sigma_i\}$ be the ReLU basis and $\{\varphi_i\}$ the hat basis on the same grid. Show that there exists a banded invertible matrix $C \in \mathbb{R}^{n\times n}$ such that

$$(\varphi_1(x), \ldots, \varphi_n(x))^\top = C\,(\sigma_1(x), \ldots, \sigma_n(x))^\top.$$

Denote by M the ReLU mass matrix and by M_{hat} the hat mass matrix. Prove that

$$M = C^{-1} M_{\text{hat}} C^{-\top}.$$

6 Suggested reading

Some content from this chapter is discussed in detail in [6, 12]. For introductions to machine learning and neural networks, we recommend APM 51053, CSC 51054, and the books [1, 4]. For optimization, see APM 43035 and [10, 11]; in particular, [10, Thm. 2.1.5] refines Theorem 1.5 by relaxing $h \leq 1/L$ to $h \leq 2/(C+L)$. Convergence results for stochastic gradient descent [14] and adaptive variants such as Adam [8] are discussed in [2, 3, 5, 12], with Theorem 1.8 paralleling [2, Thm. 4.6], [5, Thm. 3.1], and [12, Thm. 10.20]. For backpropagation [15], see [6, 12]. Beyond the McCulloch–Pitts model [9], important architectures include residual, recurrent, and convolutional networks, discussed in APM 51053, CSC 51054, and [4].

Table 1: *TL;DR table of 15 favorite references. Courses, books, and key papers grouped by topic.*

	Courses @ X	Books	Key papers
Neural networks	APM 51053 & CSC 51054	[1, 4]	[6, 9, 12]
Classical optimization	APM 43035	[10, 11]	–
SGD	APM 43035	–	[2, 3, 5, 8, 12]
Backpropagation	APM 43035	–	[6, 12, 15]
Spectral bias	–	–	[7, 13]

References

[1] M. Anthony and P. L. Barlett, *Neural Network Learning: Theoretical Foundations*, Cambridge University Press, Cambridge, 2009.

[2] L. Bottou, F. E. Curtis, and J. Nocedal, *Optimization methods for large-scale machine learning*, SIAM Rev., 60 (2018), pp. 223–311.

[3] G. Garrigos and R. M. Gower, *Handbook of Convergence Theorems for (Stochastic) Gradient Methods*, arXiv preprint, 2301.11235 (2023).

[4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, Cambridge, 2016.

[5] R. M. Gower, N. Loizou, X. Qian, A. Sailanbayev, E. Shulgin, and P. Richtárik, *SGD: General analysis and improved rates*, in ICML Proceedings, 2019.

[6] C. F. Higham and D. Higham, *Deep learning: An introduction for applied mathematicians*, SIAM Rev., 61 (2019), pp. 860–891.

[7] Q. Hong, J. W. Siegel, Q. Tan, and J. Xu, *On the activation function dependence of the spectral bias of neural networks*, arXiv preprint, 2208.04924 (2022).

[8] D. Kingma and J. Ba, *Adam: A method for stochastic optimization*, in ICLR Proceedings, 2015.

[9] W. S. McCulloch and W. Pitts, *A logical calculus of the ideas immanent in nervous activity*, Bull. Math. Biol., 5 (1943), pp. 115–133.

[10] Y. Nesterov, *Lectures on Convex Optimization*, Springer, Cham, 2nd ed., 2018.

[11] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, New York, 2nd ed., 2006.

[12] P. Petersen and J. Zech, *Mathematical Theory of Deep Learning*, arXiv preprint, 2407.18384 (2024).

[13] N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. Hamprecht, Y. Bengio, and A. Courville, *On the spectral bias of neural networks*, in ICML Proceedings, 2019.

[14] H. Robbins and S. Monroe, *A stochastic approximation method*, Ann. Math. Stat., 22 (1951), pp. 400–407.

[15] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, *Learning representations by back-propagating errors*, Nature, 323 (1986), pp. 533–536.